

Creating and Integrating a CPU Usage KPI in O-RAN Systems

Professors : **Serge FDIDA, Mai Trang NGUYEN**

Lab Assistant : **Taha Mohsen Zaidi**

Etudiants : **Emilie LIN, Eric ZHOU, Paul XU, Ilyas Benyahia**

Table des matières

1 Repository	2
2 Introduction	2
3 Objectives	2
4 Tools & Software	3
5 Technical Background	3
5.1 FlexRIC and OpenAirInterface	3
5.2 xApps	4
5.3 E2 Interface and Service Models	4
6 Project setup	4
7 CPU Metric Processing and Usage	9
7.1 Metric Definition and Objective	9
7.2 Data Collection and Computation	10
7.3 Interpretation of the Metric Value	10
8 Implementation	11
9 Results	13
9.1 Experimental Observations	13
9.2 Analysis	16

1 Repository

All code modifications are located in the 5g-workspace subfolder of the following repository : https://github.com/DHinode/cell_oran_kpi

2 Introduction

O-RAN enables open, flexible, and intelligent control of Radio Access Networks through standardized interfaces and the RAN Intelligent Controller (RIC). By decoupling hardware and software components and exposing RAN functionalities through open interfaces, O-RAN facilitates advanced monitoring and optimization via software-based applications. While existing Key Performance Measurements (KPMs) mainly focus on radio and traffic-related indicators, visibility into the computational resources of RAN nodes remains limited. As modern 5G RAN implementations increasingly rely on software processing running on general-purpose hardware, the computational load of RAN nodes becomes a critical factor affecting latency, throughput, and overall system stability. In particular, CPU overload at the gNB can degrade radio performance and limit the effectiveness of real-time control applications, highlighting the need for computational resource awareness within the O-RAN framework.

This project extends the FlexRIC KPM Service Model by introducing a CPU Usage Key Performance Metric for O-RAN nodes. The proposed metric enables real-time monitoring of CPU utilization via the E2 interface and allows xApps to subscribe to and process computational performance data. The implementation includes modifications to the KPM Service Model, updates to E2 nodes, and the development of an xApp for metric consumption.

The system is deployed using a hybrid setup with a containerized 5G Core Network and bare-metal RAN and FlexRIC components, providing a realistic and low-latency environment for testing and validating the proposed CPU usage metric.

3 Objectives

1. Create a Custom Metric :

Define and implement a new Key Performance Metric (KPM) to monitor the CPU utilization of O-RAN nodes, using standard system monitoring tools / APIs.

2. Extend the KPM Model :

Modify the existing KPM service model in FlexRIC to incorporate the new metric, ensuring compatibility and integration.

3. Update E2 Nodes :

Configure E2 nodes to support and report the newly created metric to the FlexRIC

controller.

4. **Develop an xApp :**

Build and deploy an xApp that subscribes to the new metric, processes the data, and takes actions based on the analysis.

5. **Test and Validate :**

Deploy the enhanced system in a FlexRIC environment to test and verify the accuracy and reliability of the new metric.

4 Tools & Software

- **Docker-Compose :** For deploying containerized 5G Core Network (CN) components.
- **OpenAirInterface (OAI) :** For implementing and testing the 5G RAN.
- **FlexRIC :** For extending the KPM service model and developing the xApp.
- **Bare-Metal Deployment :** For deploying the RAN and FlexRIC controller to ensure high performance in real-world scenarios.

5 Technical Background

5.1 FlexRIC and OpenAirInterface

FlexRIC is an open-source implementation of the O-RAN Near-RT RIC designed for low-latency and resource-efficient operation. Its modular architecture implements Service Models as shared dynamic libraries, enabling flexible extension and minimizing coupling between components. FlexRIC also introduces the E42 interface, which enhances communication between the Near-RT RIC and xApps and simplifies subscription and control procedures.

OpenAirInterface (OAI) is an open-source platform implementing 4G and 5G RAN components. In this project, OAI is used to implement the 5G gNB with an integrated E2 Agent, allowing the gNB to function as an E2 node capable of reporting KPM measurements to the Near-RT RIC.

The combination of FlexRIC and OAI provides an open and realistic O-RAN-compliant environment for prototyping and validating extensions to the KPM Service Model. This project leverages this environment to introduce a CPU usage KPM, enabling visibility into the computational state of the gNB and supporting CPU-aware monitoring through xApps.

5.2 xApps

xApps are software applications running on the Near-RT RIC that monitor and control RAN functions using data exposed through the E2 interface. They operate on time scales between approximately 10 ms and 1 s and enable data-driven optimization of RAN performance through standardized Service Models.

5.3 E2 Interface and Service Models

The E2 interface is a key open interface connecting the Near-RT RIC with RAN nodes, such as gNBs, CUs, and DUs. It enables both the collection of performance measurements and the execution of control actions. Each RAN node exposes its capabilities through an E2 Agent, which handles E2 signaling and communication with the RIC using the E2 Application Protocol (E2AP).

RAN functionalities are exposed via Service Models (E2SMs), which define the data structures and procedures used for monitoring and control. The Key Performance Measurement (KPM) Service Model allows xApps to subscribe to performance metrics that are periodically or conditionally reported by E2 nodes.

6 Project setup

Before going further with the project, we need to set up the 5G Core Network and launch the gNB, the UE, and the FlexRIC controller. Then, we need to open several terminals to ping each end of the network in order to test reachability.

- Build OAI with E2 Agent

```
$ Build OAI
```

```
cd ~/cell_oran_kpi/5g-workspace/oai/cmake_targets  
sudo ./build_oai -I -w SIMU --gNB --nrUE --build-e2 --ninja
```

```
-- Check if /opt/asnic/bin/asnic supports -no-gen-UPER
-- Check if /opt/asnic/bin/asnic supports -no-gen-JER
-- Check if /opt/asnic/bin/asnic supports -no-gen-BER
-- Check if /opt/asnic/bin/asnic supports -no-gen-OER
-- CMAKE_BUILD_TYPE is RelWithDebInfo
-- CPU architecture is x86_64
-- AVX512 intrinsics are OFF
-- AVX2 intrinsics are ON
-- Selected E2AP_VERSION: E2AP_V2
-- Selected KPM Version: KPM_V2_03
-- LTTNG support disabled
-- Checking for module 'libcap'
--   No package 'libcap' found
-- Checking for module 'cblas'
--   No package 'cblas' found
-- No Doxygen documentation requested
-- No Support for Aerial
-- E2AP submodule detected.
-- Selected LIBRARY TYPE: STATIC
-- Selected E2AP_ENCODING: ASN
-- Selected KPM_SM_ENCODING: ASN
-- Selected RC_SM_ENCODING: ASN
-- Selected MAC_SM_ENCODING: PLAIN
-- Selected RLC_SM_ENCODING: PLAIN
-- Selected PDCP_SM_ENCODING: PLAIN
-- Selected SLICE_SM_ENCODING: PLAIN
-- Selected GTP_SM_ENCODING: PLAIN
-- Selected SM_ENCODING: PLAIN
-- Configuring done
-- Generating done
-- Build files have been written to: /home/student/cell_oran_kpi/5g-workspace/oai/cmake_targets/ran_build/build
cd /home/student/cell_oran_kpi/5g-workspace/oai/cmake_targets/ran_build/build
Running "cmake --build . --target rfsimulator nr-uesoftmodem params_libconfig coding rfsimulator dfts -- -j8"
log file for compilation is being written to: /home/student/cell_oran_kpi/5g-workspace/oai/cmake_targets/log/all.txt
rfsimulator nr-uesoftmodem params_libconfig coding rfsimulator dfts compiled
BUILD SHOULD BE SUCCESSFUL
```

Figure 1 : Build OAI

- Build FlexRIC

\$ Build FlexRIC

```
cd ~/cell_oran_kpi/5g-workspace/flexric/
mkdir build && cd build && cmake .. && make -j8
```

```
[ 99%] Linking C executable xapp_tc_osi_code1
[ 99%] Building C object examples/xApp/c/kpm_rc/CMakeFiles/xapp_kpm_rc.dir/_/_/_/src/util/alg_ds/alg_defer.c.o
[ 99%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/rnd/fill_rnd_data_gtp.c.o
[ 99%] Linking C executable xapp_kpm_rc
[ 99%] Built target xapp_tc_partition
[ 99%] Building C object src/xApp/swig/CMakeFiles/xapp_sdk.dir/_/_/_/sm/pdcp_sm/ie/pdcp_data_ie.c.o
[ 99%] Building C object src/xApp/swig/CMakeFiles/xapp_sdk.dir/_/_/_/sm/rlc_sm/ie/rlc_data_ie.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/rnd/fill_rnd_data_tc.c.o
[100%] Building C object src/xApp/swig/CMakeFiles/xapp_sdk.dir/_/_/_/sm/slice_sm/ie/slice_data_ie.c.o
[100%] Building C object src/xApp/swig/CMakeFiles/xapp_sdk.dir/_/_/_/sm/tc_sm/ie/tc_data_ie.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/rnd/fill_rnd_data_mac.c.o
[100%] Built target xapp_tc_een
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/rnd/fill_rnd_data_rlc.c.o
[100%] Built target xapp_tc_osi_code1
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/rnd/fill_rnd_data_pdcpc.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/rnd/fill_rnd_data_kpm.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/rnd/fill_rnd_data_rc.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/rnd/fill_rnd_data_slice.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/rnd/fill_rnd_data_e2_setup_req.c.o
[100%] Built target xapp_kpm_rc
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/src/sm/kpm_sm/kpm_sm_v02_03/ie/kpm_data_ie.c.o
[100%] Building C object src/xApp/swig/CMakeFiles/xapp_sdk.dir/_/_/_/sm/gtp_sm/ie/gtp_data_ie.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/src/sm/mac_sm/ie/mac_data_ie.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/src/sm/rlc_sm/ie/rlc_data_ie.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/src/sm/pdcp_sm/ie/pdcp_data_ie.c.o
[100%] Building C object src/xApp/swig/CMakeFiles/xapp_sdk.dir/_/_/_/util/byte_array.c.o
[100%] Building C object src/xApp/swig/CMakeFiles/xapp_sdk.dir/_/_/_/util/alg_ds/alg_eq_float.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/src/sm/tc_sm/ie/tc_data_ie.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/src/sm/slice_sm/ie/slice_data_ie.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/src/sm/gtp_sm/ie/gtp_data_ie.c.o
[100%] Building C object test/agent-ric-xapp/CMakeFiles/test_ag_ric_xapp.dir/_/_/_/src/util/alg_ds/alg_defer.c.o
[100%] Linking C executable test_ag_ric_xapp
[100%] Built target test_ag_ric_xapp
[100%] Linking CXX shared module xapp_sdk.so
[100%] Built target xapp_sdk
```

Figure 2 : Build FlexRIC

- Installation of Service Models (SMs)

```
$ Service Models

cd ~/cell_oran_kpi/5g-workspace/flexric/build
sudo make install
```

```
Install the project...
-- Install configuration: "Debug"
-- Installing: /usr/local/lib/flexric/libmac_sm.so
-- Installing: /usr/local/lib/flexric/librlc_sm.so
-- Installing: /usr/local/lib/flexric/libpdcp_sm.so
-- Installing: /usr/local/lib/flexric/libslice_sm.so
-- Installing: /usr/local/lib/flexric/libtc_sm.so
-- Installing: /usr/local/lib/flexric/libgtcp_sm.so
-- Installing: /usr/local/lib/flexric/libkpm_sm.so
-- Installing: /usr/local/lib/flexric/librc_sm.so
-- Up-to-date: /usr/local/etc/flexric/flexric.conf
```

Figure 3 : Service Models Installation

- Start the 5G Core Network with docker-compose

```
docker compose -f
/home/student/cell_oran_kpi/5g-workspace/oai-cn5g/docker-compose.yaml up -d
```

```
student@lab-vm-5:~$ docker compose -f ~/cell_oran_kpi/5g-workspace/oai-cn5g/docker-compose.yaml up -d
[+] up 11/11
✓ Network oai-cn5g-public-net created 0.1s
✓ Container oai-ext-dn created 0.2s
✓ Container oai-nrf created 0.2s
✓ Container ims created 0.2s
✓ Container mysql created 0.2s
✓ Container oai-udr created 0.1s
✓ Container oai-udm created 0.1s
✓ Container oai-ausf created 0.1s
✓ Container oai-amf created 0.2s
✓ Container oai-upf created 0.1s
```

Figure 4 : 5g Core Network deployment

```
student@lab-vm-5:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                NAMES
f49153b04465   oaisoftwarealliance/oai-upf:develop /bin/bash -c 'set -e; ...' 22 minutes ago Up 22 minutes (healthy) 2152/udp, 8086/udp, 5342-5344/tcp   oai-upf
f20a904c4c65   oaisoftwarealliance/oai-amf:develop /bin/bash -c 'set -e; ...' 22 minutes ago Up 22 minutes (healthy) 88/tcp, 5342-5344/tcp, 8088/tcp, 9998/tcp, 38412/sctp oai-amf
23b0d3debcu    oaisoftwarealliance/oai-amf:develop /bin/bash -c 'set -e; ...' 22 minutes ago Up 22 minutes (healthy) 88/tcp, 5342-5344/tcp, 8088/tcp, 9998/tcp, 38412/sctp oai-amf
3d05a26c708    oaisoftwarealliance/oai-ausf:develop /bin/bash -c 'set -e; ...' 22 minutes ago Up 22 minutes (healthy) 88/tcp, 5342-5344/tcp, 8088/tcp   oai-ausf
2a3a9c9f935    oaisoftwarealliance/oai-udm:develop /bin/bash -c 'set -e; ...' 22 minutes ago Up 22 minutes (healthy) 88/tcp, 5342-5344/tcp, 8088/tcp   oai-udm
6e0dfecc6344   oaisoftwarealliance/oai-udr:develop /bin/bash -c 'set -e; ...' 22 minutes ago Up 22 minutes (healthy) 88/tcp, 8080/tcp                 oai-udr
7c2a993a55f    oaisoftwarealliance/oai-udr:develop /bin/bash -c 'set -e; ...' 22 minutes ago Up 22 minutes (healthy) 88/tcp, 5342-5344/tcp, 8088/tcp   oai-udr
c78cc0ff536    oaisoftwarealliance/ims:latest      *asterisk -fp*           22 minutes ago Up 22 minutes (healthy) 3386/tcp, 33860/tcp              ims
889280e200     mysql:8.0                           *docker-entrypoint.s.*   22 minutes ago Up 22 minutes (healthy)                                mysql
1c6f2a9718fa   oaisoftwarealliance/trf-gen-cn5g:jammy /bin/bash -c 'set -e; ...' 22 minutes ago Up 22 minutes (healthy)                                oai-ext-dn
```

Figure 5 : Checking if the 5G CN is deployed

- Start the gNB in terminal 1 :

```
$ gNB
```

```
cd ~/cell_oran_kpi/5g-workspace/oai/cmake_targets/ran_build/build
sudo ./nr-softmodem -O ../../../../targets/PROJECTS/GENERIC-NR-5GC/CONF/gnb.sa.band78.fr1.106PRB.usrpb210.conf --gNBs
.[0].min_rxtxttime 6 --rfsim --sa
```

```
[PHY] RC.nb_nr_cc[inst:0]:0x7fb31342e010
[PHY] [gNB 0] phy_init_nr_gnb() About to wait for gNB to be configured
shlib_path libdfts.so
[LOADER] library libdfts.so successfully loaded
shlib_path libldpc.so
[LOADER] library libldpc.so successfully loaded
shlib_path libldpc.so
[LOADER] library libldpc.so has been loaded previously, reloading function pointers
[LOADER] library libldpc.so successfully loaded
[PHY] Initialise nr transport
[PHY] Mapping RX ports from 1 RUs to gNB 0
[PHY] gNB->num RU:1
[PHY] Attaching RU 0 antenna 0 to gNB antenna 0
[UTIL] threadCreate() for Tpool0_1: creating thread with affinity ffffffff, priority 97
[UTIL] threadCreate() for Tpool1_1: creating thread with affinity ffffffff, priority 97
[UTIL] threadCreate() for Tpool2_1: creating thread with affinity ffffffff, priority 97
[UTIL] threadCreate() for Tpool3_1: creating thread with affinity ffffffff, priority 97
[UTIL] threadCreate() for Tpool4_1: creating thread with affinity ffffffff, priority 97
[UTIL] threadCreate() for Tpool5_1: creating thread with affinity ffffffff, priority 97
[UTIL] threadCreate() for Tpool6_1: creating thread with affinity ffffffff, priority 97
[UTIL] threadCreate() for Tpool7_1: creating thread with affinity ffffffff, priority 97
[UTIL] threadCreate() for L1_rx_thread: creating thread with affinity ffffffff, priority 97
[UTIL] threadCreate() for L1_tx_thread: creating thread with affinity ffffffff, priority 97
[UTIL] threadCreate() for L1_stats: creating thread with affinity ffffffff, priority 1
ALL RUs ready - ALL gNBs ready
Sending sync to all threads
Entering ITTI signals handler
TYPE <CTRL-C> TO TERMINATE
waiting for sync (L1_stats_thread,0/0x5632bfa7e0a8,0x5632c032d740,0x5632c032d700)
got sync (L1_stats_thread)
got sync (ru_thread)
[PHY] RU 0 rf device ready
[PHY] RU 0 RF started opp_enabled 0
[HW] No connected device, generating void samples...
```

Figure 6 : gNB start

- Start the UE in terminal 2 :

```
$ UE
```

```
cd ~/cell_oran_kpi/5g-workspace/oai/cmake_targets/ran_build/build
sudo ./nr-uesoftmodem -r 106 --numerology 1 --band 78 -C
3619200000 --rfsim --sa --uicc0.imsi 0010100000000001 --
rfsimulator.serveraddr 127.0.0.1
```

```
[NR_PHY] =====
[NR_PHY] [UE 0] Harq round stats for Downlink: 146/0/0
[NR_PHY] =====
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] =====
[NR_PHY] [UE 0] Harq round stats for Downlink: 146/0/0
[NR_PHY] =====
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] =====
[NR_PHY] [UE 0] Harq round stats for Downlink: 147/0/0
[NR_PHY] =====
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
[NR_PHY] [UE 0] RSRP = -42 dBm
```

Figure 7 : Starting the UE

- Start the near-RT RIC in terminal 3 :

```
./cell_oran_kpi/5g-workspace/flexric/build/examples/ric/nearRT-RIC
```

```
student@lab-vm-5:~/cell_oran_kpi/5g-workspace$ ./flexric/build/examples/ric/nearRT-RIC
[UTIL]: Setting the config -c file to /usr/local/etc/flexric/flexric.conf
[UTIL]: Setting path -p for the shared libraries to /usr/local/lib/flexric/
[NEAR-RIC]: nearRT-RIC IP Address = 127.0.0.1, PORT = 36421
[NEAR-RIC]: Initializing
[NEAR-RIC]: Loading SM ID = 144 with def = PDCP_STATS_V0
[NEAR-RIC]: Loading SM ID = 146 with def = TC_STATS_V0
[NEAR-RIC]: Loading SM ID = 143 with def = RLC_STATS_V0
[NEAR-RIC]: Loading SM ID = 145 with def = SLICE_STATS_V0
[NEAR-RIC]: Loading SM ID = 2 with def = ORAN-E2SM-KPM
[NEAR-RIC]: Loading SM ID = 148 with def = GTP_STATS_V0
[NEAR-RIC]: Loading SM ID = 142 with def = MAC_STATS_V0
[NEAR-RIC]: Loading SM ID = 3 with def = ORAN-E2SM-RC
[iApp]: Initializing ...
[iApp]: nearRT-RIC IP Address = 127.0.0.1, PORT = 36422
[NEAR-RIC]: Initializing Task Manager with 2 threads
[E2AP]: E2 SETUP-REQUEST rx from PLMN 1. 1 Node ID 3584 RAN type ngran_gNB
[NEAR-RIC]: Accepting RAN function ID 2 with def = ORAN-E2SM-KPM
[NEAR-RIC]: Accepting RAN function ID 3 with def = ORAN-E2SM-RC
[NEAR-RIC]: Accepting RAN function ID 142 with def = MAC_STATS_V0
[NEAR-RIC]: Accepting RAN function ID 143 with def = RLC_STATS_V0
[NEAR-RIC]: Accepting RAN function ID 144 with def = PDCP_STATS_V0
[NEAR-RIC]: Accepting RAN function ID 145 with def = SLICE_STATS_V0
[NEAR-RIC]: Accepting RAN function ID 146 with def = TC_STATS_V0
[NEAR-RIC]: Accepting RAN function ID 148 with def = GTP_STATS_V0
```

Figure 8 : Starting the near-RT RIC

- Ping between UE and network in terminal 4 :
The environment is successfully build as shown here

```
ping -c 3 192.168.70.135 -I oaitun_ue1
```

```
student@lab-vm-5:~/cell_oran_kpi/5g-workspace/oai/cmake_targets/ran_build/build$ ping -c 3 192.168.70.135 -I oaitun_ue1
PING 192.168.70.135 (192.168.70.135) from 10.0.0.2 oaitun_ue1: 56(84) bytes of data.
64 bytes from 192.168.70.135: icmp_seq=1 ttl=63 time=17.0 ms
64 bytes from 192.168.70.135: icmp_seq=2 ttl=63 time=20.2 ms
64 bytes from 192.168.70.135: icmp_seq=3 ttl=63 time=16.4 ms

--- 192.168.70.135 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 16.354/17.839/20.165/1.665 ms
```

Figure 9 : Uplink UE to Network

```
sudo docker exec -it oai-ext-dn ping -c 3 10.0.0.2
```

```
student@lab-vm-5:~/cell_oran_kpi/5g-workspace/oai/cmake_targets/ran_build/build$ sudo docker exec -it oai-ext-dn ping -c 3 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=63 time=18.5 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=63 time=19.8 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=63 time=21.1 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 18.486/19.799/21.127/1.078 ms
```

Figure 10 : Downlink ping Network to UE

7 CPU Metric Processing and Usage

This section details the implementation of the new performance metric integrated into the O-RAN architecture to monitor computing resources.

7.1 Metric Definition and Objective

The gNB_CPU_Load metric was designed to measure the actual CPU load consumed by the base station (gNB) process. Unlike a system-wide measurement, this metric specifically focuses on resource usage by the OAI protocol layers (PHY, MAC, RLC, PDCP, and RRC). The primary objective is to enable the controller (RIC) to take proactive actions, such as :

- **Overload detection** : Identifying whether signal processing functions are saturating CPU cores.
- **Load balancing** : Redirecting new user equipments (UEs) to another cell if the CPU load exceeds a predefined threshold.
- **Energy optimization** : Reducing allocated resources during periods of low traffic to save energy.

7.2 Data Collection and Computation

The computation of the `gNB_CPU_Load` metric relies on the exploitation of Linux kernel data structures through the E2 agent integrated into the OpenAirInterface source code. This approach enables an accurate estimation of the CPU resources consumed specifically by the gNB process.

To provide a meaningful CPU load percentage, the agent computes a differential between two measurement intervals (t_1 and t_2). Conceptually, the CPU load corresponds to the ratio between the CPU time consumed by the gNB process and the real elapsed time between two measurements :

$$\text{CPU Load} = \frac{\Delta \text{CPU Time}}{\Delta \text{Wall_Clock_Time}} \times 100 \quad (1)$$

This conceptual formulation can be expressed more precisely by accounting for both user-mode execution time and kernel-mode system calls. The resulting detailed equation is given by :

$$\text{CPU Load} = \frac{(U_2 + S_2) - (U_1 + S_1)}{T_2 - T_1} \times 100 \quad (2)$$

where :

- U represents the user CPU time (`ru_utime`);
- S represents the system CPU time (`ru_stime`);
- T represents the wall-clock elapsed time between two measurement instants.

The overall process consists of three key steps :

1. **System data retrieval** : The E2 agent invokes the system call `getrusage(RUSAGE_SELF)` in the file `ran_func_kpm_subs.c` to retrieve the CPU time consumed by the gNB process.
2. **Differential computation** : The current measurement is compared with the previous one to obtain a real-time estimation of the CPU load.
3. **E2 transmission** : The computed CPU load value is encapsulated into a KPM indication message (Service Model ID 2) and transmitted to the FlexRIC controller via the E2 interface.

7.3 Interpretation of the Metric Value

It is crucial to understand that the reported value may exceed 100%.

Multi-core interpretation

In a Linux environment, a load of 100% corresponds to the full utilization of a single

CPU core. Since OpenAirInterface is a multi-threaded application leveraging multiple cores for parallel radio signal processing, a value of 157.77% indicates that the gNB is using approximately 1.6 CPU cores simultaneously.

Performance analysis

- **Low value (e.g., < 50%)** : The gNB is idle or handling very few UEs.
- **Medium value (e.g., 100%–200%)** : Normal operation with active data traffic.
- **Critical value** : If the metric approaches the maximum number of available CPU cores on the host machine, processing delays (latency) may occur on the radio interface.

8 Implementation

- Path : 5g-workspace/oai/openair2/E2AP/RAN_FUNCTION/O-RAN/ran_func_kpm_subs.c
- Add the function that calculates CPU usage using Linux system calls.

ran_func_kpm_subs.c

```
1 #include <sys/time.h>
2 #include <sys/resource.h>
3
4 static struct rusage last_usage;
5 static struct timeval last_time;
6 static int first_run = 1;
7
8 static meas_record_list_t fill_CPU_Metric(uint32_t
    gran_period_ms, cud_u_ue_info_t *ue_info, const size_t
    ue_idx) {
9     meas_record_list_t meas_record = {0};
10    meas_record.value = REAL_MEAS_VALUE;
11
12    struct rusage current_usage;
13    struct timeval current_time;
14
15    getrusage(RUSAGE_SELF, &current_usage);
16    gettimeofday(&current_time, NULL);
17
18    if (first_run) {
19        last_usage = current_usage;
20        last_time = current_time;
21        first_run = 0;
22        meas_record.real_val = 0.0;
23        return meas_record;
24    }
25
26    double user_delta = (double)(current_usage.ru_utime.
```

```
27     tv_sec = last_usage.ru_utime.tv_sec) +  
           (double)(current_usage.ru_utime.  
28     tv_usec = last_usage.ru_utime.tv_usec) / 1000000.0;  
29  
     double sys_delta = (double)(current_usage.ru_stime.  
30     tv_sec = last_usage.ru_stime.tv_sec) +  
           (double)(current_usage.ru_stime.  
31     tv_usec = last_usage.ru_stime.tv_usec) / 1000000.0;  
32  
     double time_delta = (double)(current_time.tv_sec -  
33     last_time.tv_sec) +  
           (double)(current_time.tv_usec -  
34     last_time.tv_usec) / 1000000.0;  
35  
     if (time_delta > 0) {  
36         meas_record.real_val = ((user_delta + sys_delta) /  
time_delta) * 100.0;  
37     } else {  
38         meas_record.real_val = 0.0;  
39     }  
40  
41     last_usage = current_usage;  
42     last_time = current_time;  
43  
44     return meas_record;  
45 }
```

- Path : 5g-workspace/oai/openair2/E2AP/RAN_FUNCTION/O-RAN/ran_func_kpm_subs.c
- Register the function in the `lst_measure` array and specify which function the metric will use.

ran_func_kpm_subs.c

```
1 static kv_measure_t lst_measure[] = {  
2     { .key = "gNB_CPU_Load", .value = fill_CPU_Metric }, //  
   registration of our metric  
3     ...  
4 };
```

- Path : 5g-workspace/oai/openair2/E2AP/RAN_FUNCTION/O-RAN/ran_func_kpm.c
- Add the metric name string so the E2 Agent advertises it.

ran_func_kpm.c

```
1 static const char* kpm_meas_gnb[] = {  
2     "DRB.PdcpSduVolumeDL",
```

```
3  "DRB.PdcpSduVolumeUL",  
4  "DRB.RlcSduDelayDl",  
5  "DRB.UETHpDl",  
6  "DRB.UETHpUl",  
7  "RRU.PrbTotDl",  
8  "RRU.PrbTotUl",  
9  "gNB_CPU_Load", // our metric name for E2 Agent to  
    advertises  
10 NULL,  
11 };
```

- Path : 5g-workspace/flexric/examples/xApp/c/monitor/xapp_kpm_moni.c
- Update the xApp by adding a print statement to the logging function for our metric.

xapp_kpm_moni.c

```
1 static void log_real_value(byte_array_t name,  
    meas_record_lst_t meas_record){  
2  ...  
3  } else if (cmp_str_ba("DRB.UETHpUl", name) == 0) {  
4    printf("DRB.UETHpUl = %.2f [kbps]\n", meas_record.  
        real_val);  
5  } else if (cmp_str_ba("gNB_CPU_Load", name) == 0){  
6    printf("gNB CPU Load = %.2f %%\n", meas_record.real_val);  
7  } else {  
8    printf("Measurement Name not yet supported\n");  
9  }
```

At this point, we must rebuild the OAI softmodem to include the new C logic in the gNB and recompile the xApp by following the same steps as described in Section 6.

9 Results

The following results demonstrate the correct reporting and consumption of the proposed CPU usage KPM, illustrated through xApp outputs and concurrent traffic measurements using iperf.

9.1 Experimental Observations

The following tests involve one client and one server within the network :

```
student@lab-vm-5:~/cell_oran_kpi/5g-workspace/flexric$ ./build/examples/xApp/c/monitor/xapp_kpm_moni
[UTIL]: Setting the config -c file to /usr/local/etc/flexric/flexric.conf
[UTIL]: Setting path -p for the shared libraries to /usr/local/lib/flexric/
[xApp]: Initializing ...
[xApp]: nearRT-RIC IP Address = 127.0.0.1, PORT = 36422
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libpdcip_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libtc_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/librlc_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libslice_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libkpm_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libgtp_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/libmac_sm.so
[E2 AGENT]: Opening plugin from path = /usr/local/lib/flexric/librc_sm.so
[NEAR-RIC]: Loading SM ID = 144 with def = PDCP_STATS_V0
[NEAR-RIC]: Loading SM ID = 146 with def = TC_STATS_V0
[NEAR-RIC]: Loading SM ID = 143 with def = RLC_STATS_V0
[NEAR-RIC]: Loading SM ID = 145 with def = SLICE_STATS_V0
[NEAR-RIC]: Loading SM ID = 2 with def = ORAN-E2SM-KPM
[NEAR-RIC]: Loading SM ID = 148 with def = GTP_STATS_V0
[NEAR-RIC]: Loading SM ID = 142 with def = MAC_STATS_V0
[NEAR-RIC]: Loading SM ID = 3 with def = ORAN-E2SM-RC
[xApp]: DB filename = /tmp/xapp_db_1767809773657060
[xApp]: E42 SETUP-REQUEST tx
[xApp]: E42 SETUP-RESPONSE rx
[xApp]: xApp ID = 7
[xApp]: Registered E2 Nodes = 1
Connected E2 nodes = 1
[xApp]: E42 RIC SUBSCRIPTION REQUEST tx RAN_FUNC_ID 2 RIC_REQ_ID 1
[xApp]: SUBSCRIPTION RESPONSE rx
[xApp]: Successfully subscribed to RAN_FUNC_ID 2
```

Figure 11 : Successful subscription to metrics

```
student@lab-vm-5:~/cell_oran_kpi/5g-workspace/cell_monitoring/targets/ran_build/build sub docker exec -it out-ec-ds iperf -s -i 1 -t 10 -B 10.0.0.2
server listening on TCP port 5001
TCP window size: 128 KByte (default)
[ 1] local 192.168.70.135 port 5001 connected with 192.168.70.134 port 39113
[ 0] Interval      Transfer      Bandwidth
[ 1] 0.0000-1.0000 sec  1290 Kbytes  10637 Kbits/sec
[ 2] 1.0000-2.0000 sec  1274 Kbytes  10439 Kbits/sec
[ 3] 2.0000-3.0000 sec  1188 Kbytes  10729 Kbits/sec
[ 4] 3.0000-4.0000 sec  1262 Kbytes  10335 Kbits/sec
[ 5] 4.0000-5.0000 sec  1279 Kbytes  10474 Kbits/sec
[ 6] 5.0000-6.0000 sec  1281 Kbytes  10407 Kbits/sec
[ 7] 6.0000-7.0000 sec  1282 Kbytes  10300 Kbits/sec
[ 8] 7.0000-8.0000 sec  1321 Kbytes  10823 Kbits/sec
[ 9] 8.0000-9.0000 sec  1281 Kbytes  10407 Kbits/sec
[10] 9.0000-10.0000 sec  1259 Kbytes  10312 Kbits/sec
[11] 0.0000-10.0000 sec  12028 Kbytes  10571 Kbits/sec
```

Figure 13 : Test iperf server side

```
student@lab-vm-5:~$ iperf -c 192.168.70.135 -i 1 -b 10M -B 10.0.0.2
Client connecting to 192.168.70.135, TCP port 5001
TCP window size: 85.0 KByte (default)
[ 1] local 10.0.0.2 port 39113 connected with 192.168.70.135 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-1.0000 sec  1.25 MBytes  10.5 Mbits/sec
[ 1] 1.0000-2.0000 sec  1.25 MBytes  10.5 Mbits/sec
[ 1] 2.0000-3.0000 sec  1.25 MBytes  10.5 Mbits/sec
[ 1] 3.0000-4.0000 sec  1.25 MBytes  10.5 Mbits/sec
[ 1] 4.0000-5.0000 sec  1.25 MBytes  10.5 Mbits/sec
[ 1] 5.0000-6.0000 sec  1.25 MBytes  10.5 Mbits/sec
[ 1] 6.0000-7.0000 sec  1.25 MBytes  10.5 Mbits/sec
[ 1] 7.0000-8.0000 sec  1.25 MBytes  10.5 Mbits/sec
[ 1] 8.0000-9.0000 sec  1.25 MBytes  10.5 Mbits/sec
[ 1] 9.0000-10.0000 sec  1.25 MBytes  10.5 Mbits/sec
[ 1] 0.0000-10.0550 sec  12.6 MBytes  10.5 Mbits/sec
```

Figure 14 : Test iperf client side

```

10 KPM ind_msg latency = 1767808017903272 [µs]
UE ID type = gNB, amf_ue_ngap_id = 1
ran_ue_id = 1
DRB.PdcpSduVolumeDL = 0 [kb]
DRB.PdcpSduVolumeUL = 0 [kb]
DRB.RlcSduDelayDL = 0.00 [µs]
DRB.UEThpDL = 0.00 [kbps]
DRB.UEThpUL = 0.00 [kbps]
RRU.PrbTotDL = 10 [PRBs]
RRU.PrbTotUL = 85 [PRBs]
gNB CPU Load = 156.25 %
[xApp]: E42 RIC SUBSCRIPTION DELETE_REQUEST tx RAN_FUNC_ID 2 RIC_REQ_ID 1
[xApp]: E42 SUBSCRIPTION DELETE_RESPONSE rx
[xApp]: Successfully stopped
Test xApp run SUCCESSFULLY

```

Figure 12 : CPU Usage xApp Result 2

The following tests involve 5 clients and one server within the network :

```

6 KPM ind_msg latency = 1768230869769180 [µs]
UE ID type = gNB, amf_ue_ngap_id = 1
ran_ue_id = 1
DRB.PdcpSduVolumeDL = 286468 [kb]
DRB.PdcpSduVolumeUL = 17 [kb]
DRB.RlcSduDelayDL = 903044.64 [µs]
DRB.UEThpDL = 85996.06 [kbps]
DRB.UEThpUL = 19.64 [kbps]
RRU.PrbTotDL = 121739 [PRBs]
RRU.PrbTotUL = 108 [PRBs]
gNB CPU Load = 186.08 %

7 KPM ind_msg latency = 1768230870771045 [µs]
UE ID type = gNB, amf_ue_ngap_id = 1
ran_ue_id = 1
DRB.PdcpSduVolumeDL = 302472 [kb]
DRB.PdcpSduVolumeUL = 6 [kb]
DRB.RlcSduDelayDL = 925083.45 [µs]
DRB.UEThpDL = 83668.66 [kbps]
DRB.UEThpUL = 7.53 [kbps]
RRU.PrbTotDL = 118399 [PRBs]
RRU.PrbTotUL = 383 [PRBs]
gNB CPU Load = 181.99 %

8 KPM ind_msg latency = 1768230871767038 [µs]
UE ID type = gNB, amf_ue_ngap_id = 1
ran_ue_id = 1
DRB.PdcpSduVolumeDL = 296675 [kb]
DRB.PdcpSduVolumeUL = 0 [kb]
DRB.RlcSduDelayDL = 951997.56 [µs]
DRB.UEThpDL = 81869.82 [kbps]
DRB.UEThpUL = 0.74 [kbps]
RRU.PrbTotDL = 115855 [PRBs]
RRU.PrbTotUL = 210 [PRBs]
gNB CPU Load = 186.74 %

9 KPM ind_msg latency = 1768230872769607 [µs]
UE ID type = gNB, amf_ue_ngap_id = 1
ran_ue_id = 1
DRB.PdcpSduVolumeDL = 295300 [kb]
DRB.PdcpSduVolumeUL = 11 [kb]
DRB.RlcSduDelayDL = 990045.95 [µs]
DRB.UEThpDL = 78450.13 [kbps]
DRB.UEThpUL = 12.58 [kbps]
RRU.PrbTotDL = 111004 [PRBs]
RRU.PrbTotUL = 422 [PRBs]
gNB CPU Load = 183.10 %
[xApp]: E42 RIC SUBSCRIPTION DELETE_REQUEST tx RAN_FUNC_ID 2 RIC_REQ_ID 1
[xApp]: E42 SUBSCRIPTION DELETE_RESPONSE rx

10 KPM ind_msg latency = 1768230873774740 [µs]
UE ID type = gNB, amf_ue_ngap_id = 1
ran_ue_id = 1
DRB.PdcpSduVolumeDL = 296329 [kb]
DRB.PdcpSduVolumeUL = 22 [kb]
DRB.RlcSduDelayDL = 895485.29 [µs]
DRB.UEThpDL = 86717.74 [kbps]
DRB.UEThpUL = 24.78 [kbps]
RRU.PrbTotDL = 122720 [PRBs]
RRU.PrbTotUL = 502 [PRBs]
gNB CPU Load = 185.89 %
[xApp]: Successfully stopped
Test xApp run SUCCESSFULLY

```

Figure 15 : xApp KPM result for the load test


```

[ 3] WARNING: ack of last datagram failed
[ 42] 61.0000-62.0000 sec 4.71 Mbytes 39.5 Mbits/sec 0.408 ms 33220/36586 (91%)
[ 49] 44.0000-45.0000 sec 4.19 Mbytes 35.1 Mbits/sec 0.136 ms 26805/29792 (90%)
[ 42] 62.0000-63.0000 sec 5.26 Mbytes 44.2 Mbits/sec 0.768 ms 32336/36091 (90%)
[ 49] 45.0000-46.0000 sec 2.54 Mbytes 21.3 Mbits/sec 0.558 ms 14826/16640 (89%)
[ 42] 63.0000-64.0000 sec 6.42 Mbytes 53.9 Mbits/sec 1.263 ms 34723/39383 (88%)
[ 49] 46.0000-47.0000 sec 4.26 Mbytes 36.6 Mbits/sec 0.046 ms 30180/33291 (91%)
[ 42] 64.0000-65.0000 sec 5.59 Mbytes 46.1 Mbits/sec 0.140 ms 33317/37241 (89%)
[ 49] 47.0000-48.0000 sec 4.64 Mbytes 38.9 Mbits/sec 0.076 ms 40908/44228 (93%)
[ 42] 65.0000-66.0000 sec 5.12 Mbytes 42.9 Mbits/sec 0.117 ms 46846/50495 (93%)
[ 49] 48.0000-49.0000 sec 4.80 Mbytes 40.3 Mbits/sec 0.053 ms 43075/46502 (93%)
[ 42] 66.0000-67.0000 sec 5.01 Mbytes 42.1 Mbits/sec 0.060 ms 40789/44305 (92%)
[ 49] 49.0000-50.0000 sec 4.25 Mbytes 35.7 Mbits/sec 0.083 ms 37171/40285 (92%)
[ 42] 67.0000-68.0000 sec 5.47 Mbytes 45.9 Mbits/sec 0.049 ms 44066/47967 (92%)
[ 49] 50.0000-51.0000 sec 4.01 Mbytes 33.6 Mbits/sec 0.095 ms 29735/32594 (91%)
[ 42] 68.0000-69.0000 sec 2.60 Mbytes 45.6 Mbits/sec 0.325 ms 17764/19619 (91%)
[ 42] 0.0000-69.4783 sec 240 Mbytes 29.4 Mbits/sec 0.325 ms 2094065/2265441 (92%)
[ 49] 0.0000-51.1201 sec 160 Mbytes 27.6 Mbits/sec 0.225 ms 1527435/1647535 (93%)
[SUM] 0.0000-423.3818 sec 2.87 Gbytes 58.2 Mbits/sec 25694522/27789342 (92%)
[SUM] 0.0000-423.3818 sec 71 datagrams received out-of-order

```

Figure 16 : server side for the iperf test

9.2 Analysis

The implementation and testing of the gNB_CPU_Load KPI demonstrate a successful extension of the O-RAN KPM Service Model, providing critical visibility into the gNB's computational state. By comparing the results across different user loads, we can analyze how the software-based RAN scales under pressure and identify the primary drivers of CPU consumption.

Table 1 : Comparison of gNB CPU Load : Single vs. Multiple Clients

Scenario	CPU Load (%)	Technical Observation
No Traffic (Idle)	≈ 150% – 155%	Baseline for PHY sync and busy-waiting.
1 Client (iperf)	156.25%	Minimal overhead for single-user data.
5 Clients (iperf)	181.99% – 186.74%	Scaling for RRC context and scheduling.

The data reveals that the transition from a single client to five concurrent clients results in a relatively small increase in CPU utilization (approximately 18% to 30% additional load). In our Linux-based environment, where 100% represents the saturation of one core, the baseline of 156.25% confirms that OpenAirInterface is a multi-threaded application utilizing roughly 1.6 cores. The fact that the load does not quintuple with five times the users indicates that the system is computationally efficient at scaling ; the additional processing is largely dedicated to managing more Radio Resource Control (RRC) contexts and the increased complexity of the MAC layer scheduler.

A particularly noteworthy observation is that the CPU load remains high (above 150%) even when no active iperf traffic is present. This "High Idle Load" is a hallmark of software-defined radio (SDR) platforms like OpenAirInterface. To maintain the 5G cell, the physical layer (PHY) must continuously process radio frames, broadcast system information, and listen for incoming signals from UEs every millisecond. Furthermore, OAI often utilizes "busy-waiting" loops to minimize processing latency, which keeps the CPU cores in a high-power state to respond instantly to radio events. Thus, the computational budget is dominated by a fixed "maintenance cost" of the radio stack, while user-specific traffic

represents a variable but smaller cost.

In summary, the gNB_CPU_Load metric provides an accurate and reactive measurement of the gNB's processing health. The results confirm that while the gNB scales well with additional users, the underlying SDR architecture requires a significant baseline of computational resources to maintain network synchronization and real-time performance.